

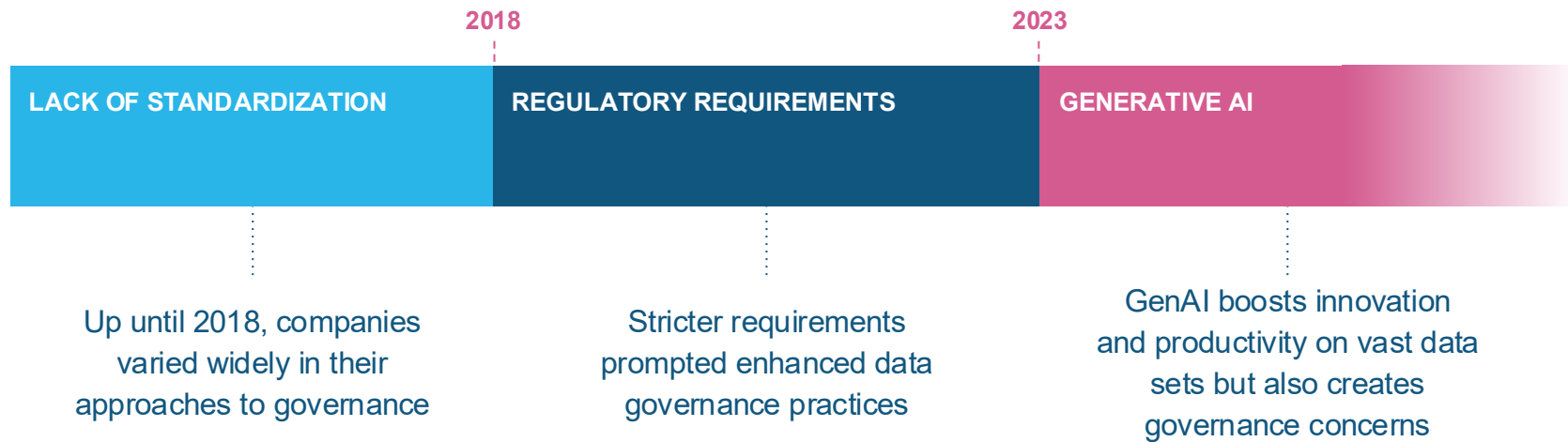


Snowflake Security & Governance

Snowflake 中国 Office Hour

2026年5月

The Importance of Governance is Magnified by the Recent Rise of GenAI



A Well-Governed Data Foundation Drives Success For Productizing AI, Applications and More



Accelerate success for AI, apps and more

Enable customers to maximize value from data pipelines, analytics, AI and app development in a governed way

Govern your data foundation

Govern and discover data, apps and models



Why Snowflake Horizon Catalog?



Centralize Threat Monitoring and Role-Based Access Controls Across Clouds

Detect and resolve threats across clouds while providing secure access to only users with the appropriate roles.



Easily Protect All Content with Granular Capabilities

Apply out-of-the-box, proven governance protections on content in Snowflake, in object storage or in open table formats, whether internal or external to your organization.

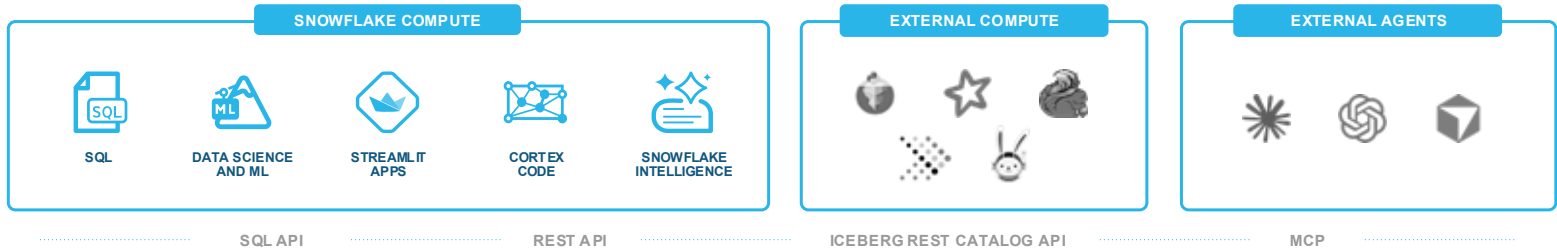


Enable Quick Discovery and Privacy-Preserving Collaboration

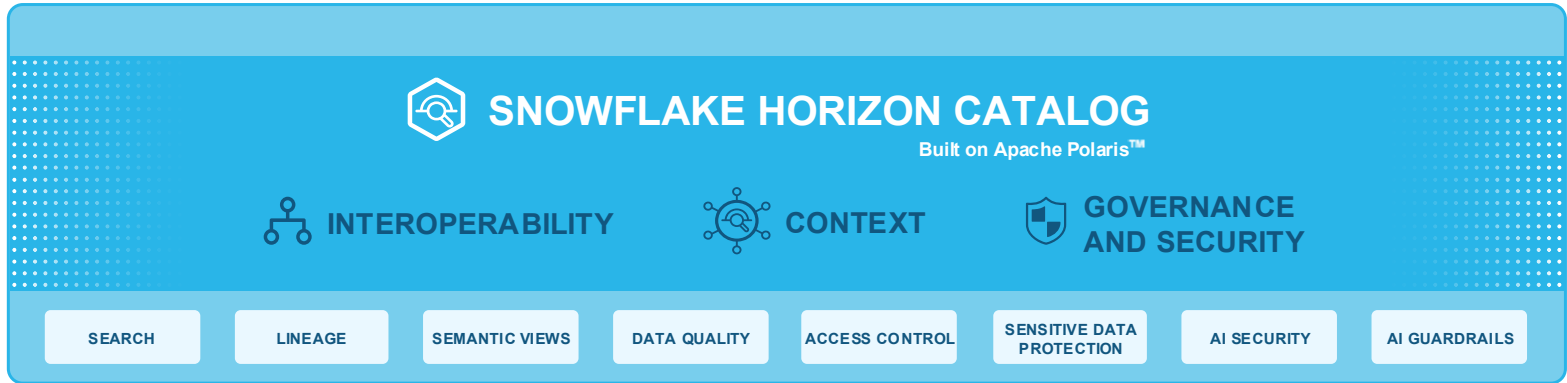
Find data, apps and models with natural language and collaborate while protecting sensitive or internal content.



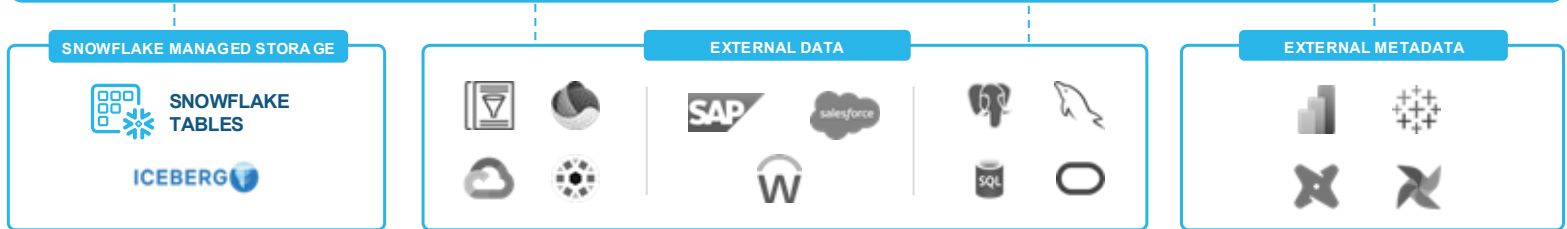
COMPUTE AND AI



CATALOG



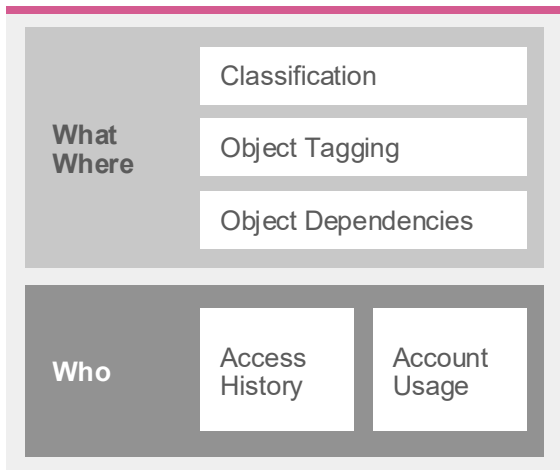
DATA AND STORAGE



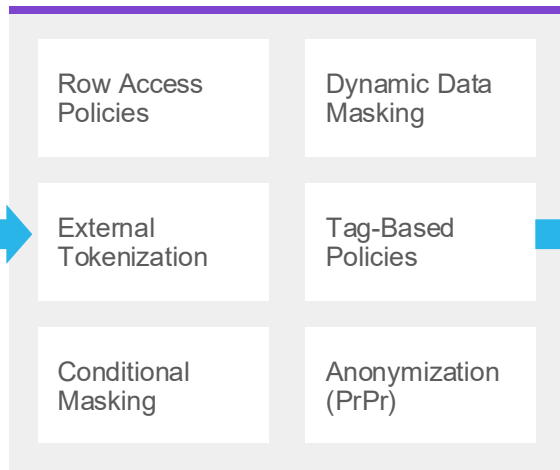
Unified, Cross-Cloud Governance



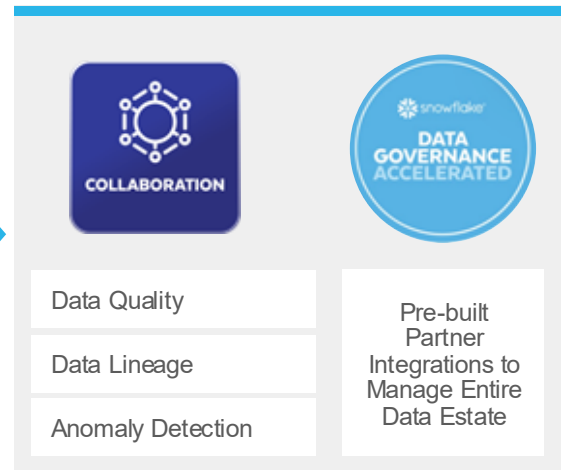
Know Your Data



Protect Your Data



Govern Your Data

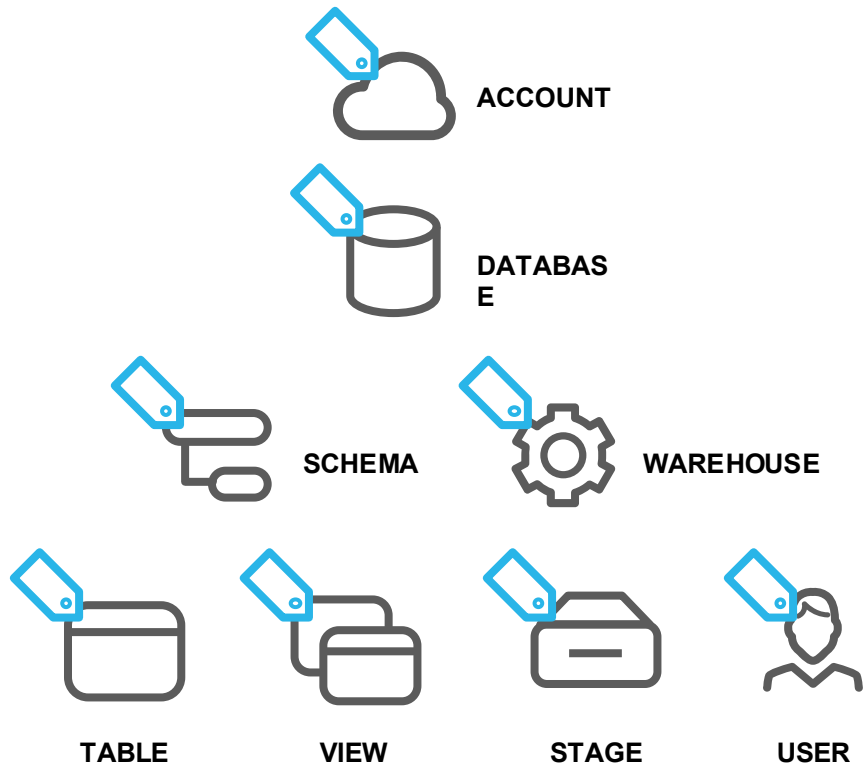


Know Your Data



Object Tagging

Track sensitive data and compute objects



Track sensitive and PII data

Track resource usage for cost visibility and attribution

Flexible privilege management options



Classification

Classify sensitive personal data



fname	gen	age	phone
Jane Doe	F	50	333-555-1236
Michael Gaines	M	75	666-666-1357
Ann Marshall	F	48	555-555-1234
John Smith	M	39	123-555-1234

Analyze table columns for sensitive personal information

Get recommended tags using built-in machine learning

Apply tags to track and audit sensitive data



Protect Your Data



Row Access Policies

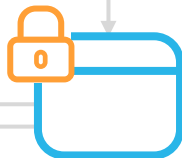
Row-level Security

Customer	Spend	Region
ACME	\$820,000	North America
Koko	\$2,100,000	North America


ROLE:
EU_NA

Customer	Spend	Region
AGM	\$5,757,000	Europe


ROLE:
EU_RL



Filter unauthorized rows at query time

Use mapping tables for authorization

Apply one policy to many tables

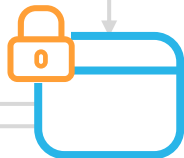


Dynamic Data Masking

Column-level Security

ID	Phone	SSN
101	***-***-5534	*****
102	***-***-3564	*****
103	***-***-9787	*****


(Authorized
Access:
Restricted Data)



ID	Phone	SSN
101	408-123-5534	111-22-3333
102	510-335-3564	222-33-4444
103	214-553-9787	333-44-5555


(Authorized
Access:
Unrestricted Data)

Dynamically mask data at query time

Centralized policy management

Apply one policy to many columns



External Tokenization

Dynamically de-tokenize data for authorized users

Externally tokenize protected data at ingest

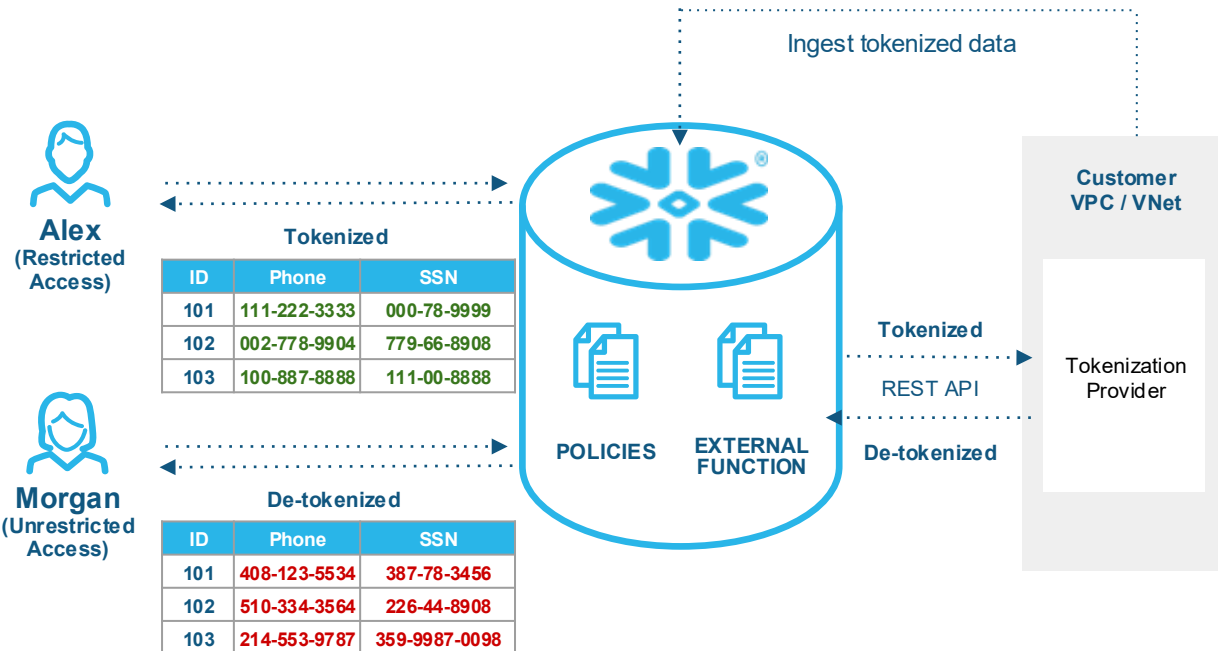
- Using tokenization provider agents on ETL tools

Dynamically de-tokenize at query time

- Call third-party service using external functions to de-tokenize data
- For unauthorized users, third-party service is not called

Policy Based Control

- Table/View owners and privileged users unauthorized by default
- Centralized policy management



Conditional Masking

Conditionally mask based on value in other column(s)

Conditionally mask protected field-level data

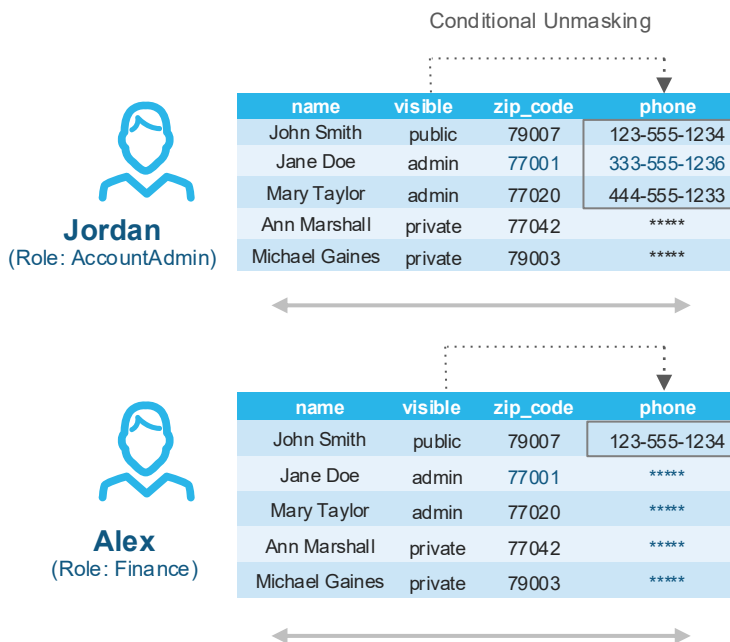
- No change to stored data
- Unmask fields for authorized users only

Extend Masking Policies

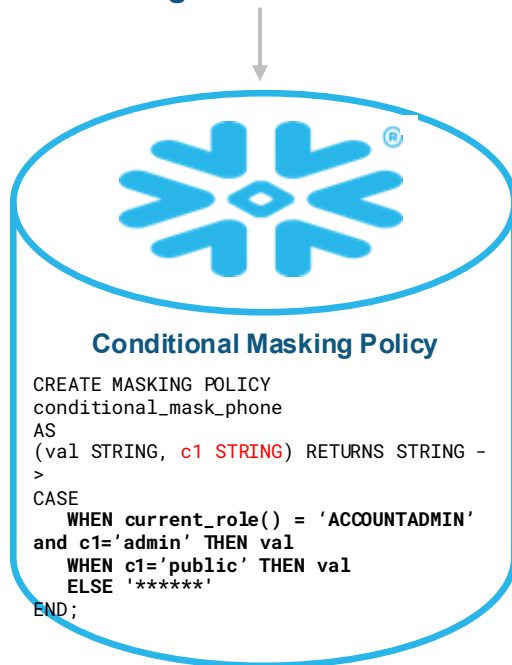
- Add authorization to existing masking policies

Ease of Management

- Prevent secure view explosion

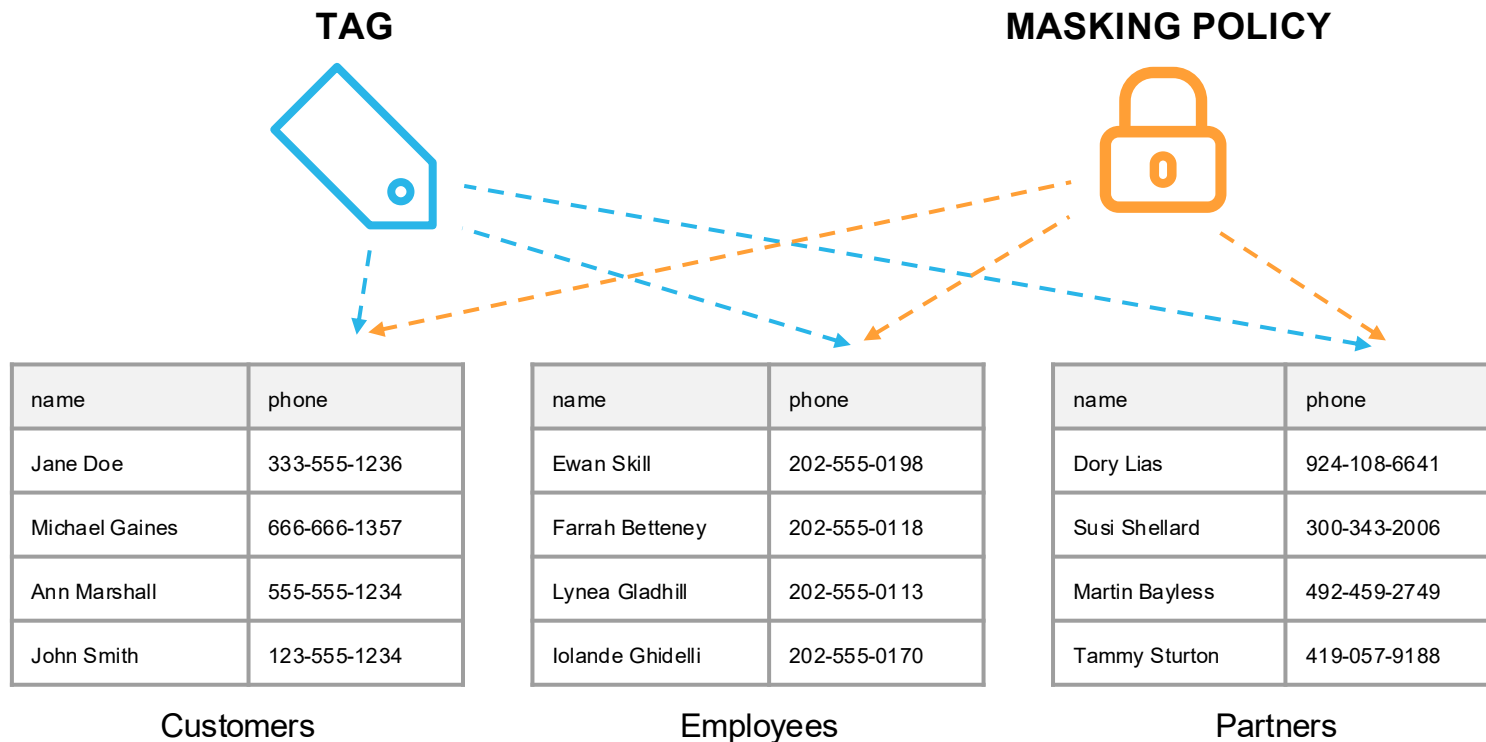


Ingest Raw Data

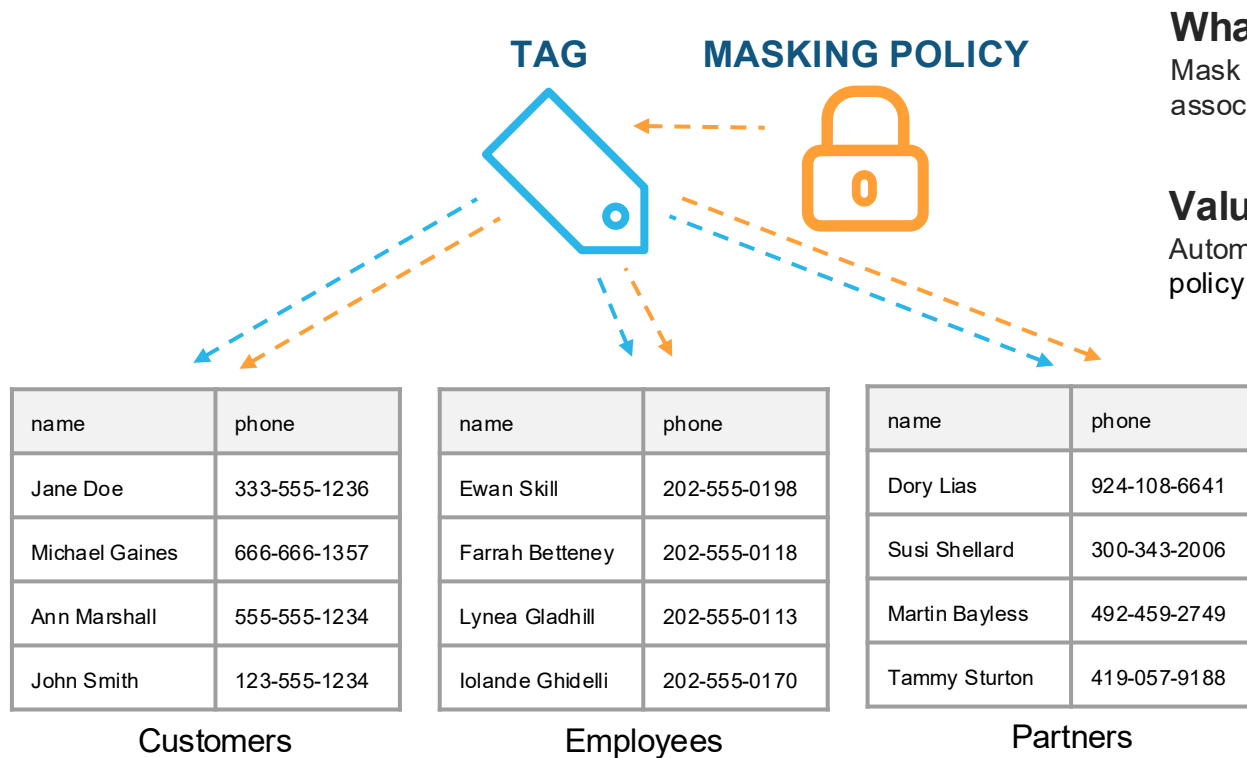


Tagging & Masking in the Past

Tags and Masking Policies assigned and applied independently



Tag-Based Masking



What is it?

Mask columns automatically based on tags associated with them

Value

Automated data protections and simplify policy change management at scale.



Govern Your Data



Access History (Reads)

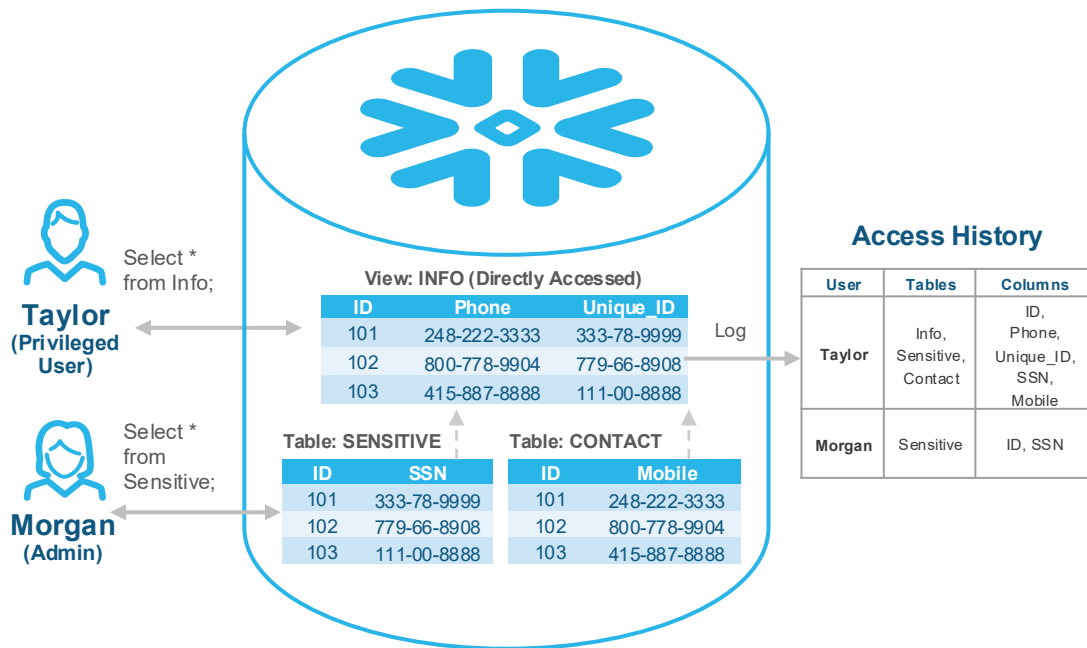
Satisfy regulatory compliance, understand usage with column-level access visibility

Satisfy regulatory compliance requirements

- Audit log of tables, views, and columns accessed by each query
- Includes base tables (i.e. tables serving a view) indirectly accessed
- Easily query historic logs for reporting and auditing purposes

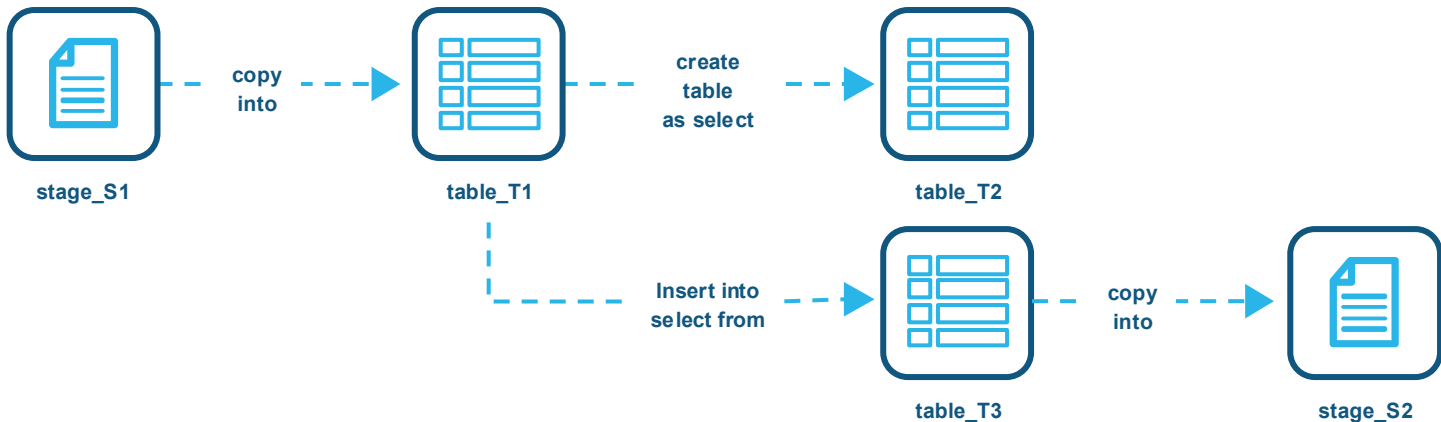
Optimize storage cost and performance

- Discover unused tables and columns for deletion or archival
- Identify your most frequently queried tables and columns for performance optimizations



Access History (Writes)

Know the lineage of data

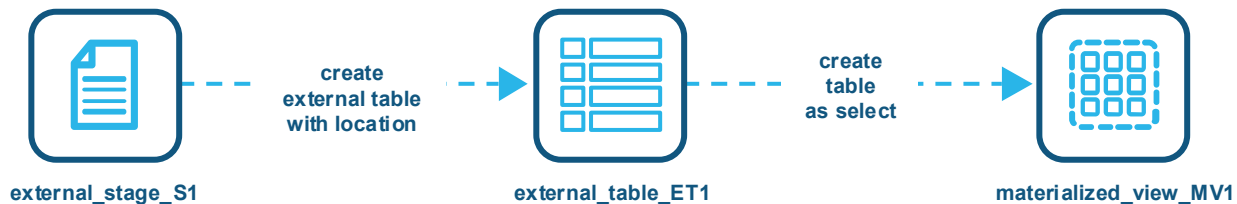


PATH	TARGET_NAME	TARGET_DOMAIN	TARGET_COLUMNS
stage_S1→table_T1	table_T1	TABLE	["CONTENT"]
stage_S1→table_T1→table_T2	table_T2	TABLE	["ID","NAME"]
stage_S1→table_T1→table_T3	table_T3	TABLE	["NAME","ID"]
stage_S1→table_T1→table_T3→stage_S2	stage_S2	STAGE	[]



Object Dependencies

Identify dependencies and downstream impact



REFERENCED_OBJECT_NAME	REFERENCED_OBJECT_DOMAIN	REFERENCING_OBJECT_NAME	TREFERENCING_OBJECT_DOMAIN
external_stage_S1	STAGE	external_table_ET1	EXTERNAL TABLE
external_table_ET1	EXTERNAL_TABLE	materialized_view_MV1	MATERIALIZED_VIEW



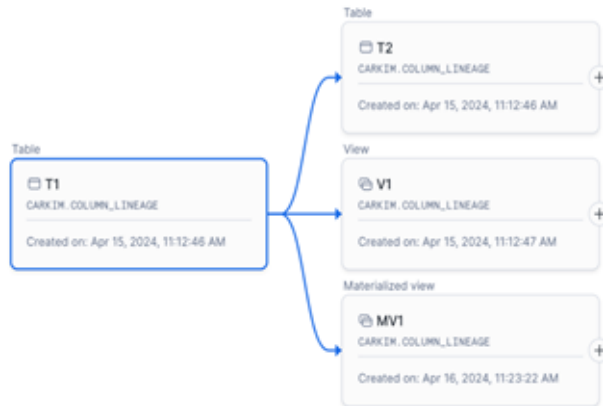
Proactively Take Control of Data Quality

Continuously Monitor Metrics



Enable high data quality and be alerted to problems; fully customizable

Analyze Root Cause with Lineage



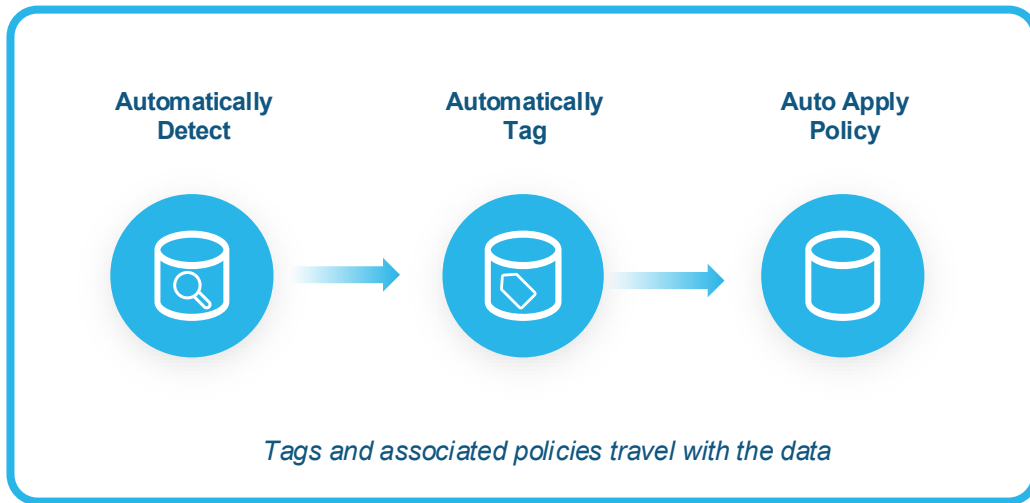
See end to end data flow, including training data for AI models.



Detect and Protect Sensitive Data

- Automatically detect sensitive data
- Tag it accurately (PII, PCI, PHI, etc.)
- Ensure tags travel with the sensitive data via Automatic Tag Propagation
- Assign fine-grained policies ensuring correct use and access
- Collaborate internally and share externally knowing sensitive data is protected
- Report compliance using interactive governance UI

Automatically Treat Sensitive Data Differently



Case Study



FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y

We start with a table

- The table has been instantiated from the encrypted, at-rest files (micro-partitions)
- The information in the table is opaque to Snowflake
- This table contains data “in the clear,” but you might load data that’s been modified in some way to protect it as well

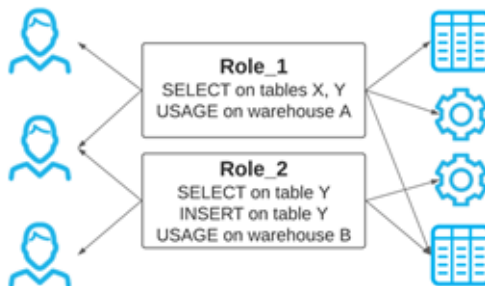


RBAC & DAC

Views & UDFs

name	age	zip_code
Polk	5-	169**
Gaines	6-	288**
Gordon	3-	851**
Hammer	6-	505**

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y

**RBAC & DAC protect the table**

- Every object in Snowflake is subject to these controls, and they are at the whole-object level
- RBAC inheritance and other RBAC features apply
- The customer controls RBAC completely
- DAC (Discretionary Access Control) applies to the role that owns the object, unless the object is subject to [Managed Access Schema](#)

You may also create Views & UDFs

- These are mostly used to redact or transform rows, columns, or even cells, and create a new object
- The new object has RBAC and DAC controls



RBAC & DAC

Views & UDFs

Column-Level Security

Row Access Policy

Conditional Policy

We can use Policy controls for Columns, Rows, and Fields

- Prevent View/UDF explosion
- Table/View owners and privileged users (such as ACCOUNTADMIN) unauthorized to data by default
- Ensure controls are applied in any context where the object's data is used

We get more ease of management

- Centrally manage policies
- Apply a single policy to multiple tables
- Built-in separation of duty: policy admins assign, and users are subject to policy controls
- All application and use is fully audited

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y



Dynamic Data Masking

RBAC & DAC

Views & UDFs

Column-Level Security

Dynamic Data Masking

We can leverage Column-level Security to dynamically mask data at query time

- No change to the stored data
- Mask or partially mask using constant value, hash, and custom functions
- Unmask for authorized users only

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y

Query results

phone	name
--5534	** masked **
--3564	** masked **
--9787	** masked **



Alex
(Unauthorized)

Query results

phone	name
408-123-5534	** masked **
510-335-3564	** masked **
214-553-9787	** masked **



Morgan
(Partially Authorized)

```
create or replace masking policy F00
as (val string) returns string ->
case
  when is_granted_to_invoker_role('SECCLEAR')
  then val
  when current_role('ONLYPART')
  then regexp_replace(val, '[0-9]', '*', 7)
  when is_role_in_session('CRYPTO')
  then decrypt_raw(val, keyval, IV, ...)
  when is_role_in_session('BESPOKE')
  then user_defined_Func(val, baz, ...)
  else '** masked **'
end;
```



RBAC & DAC

Views & UDFs

Column-Level Security

External Tokenization

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	awiufyaf873fg	Y
Polk	N	52	16971	78ybhsbcbvzd	U
Gaines	F	61	28877	984iuwrfgjffsss	Y
Gordon	M	36	85115	ciudsjhciasudg	N
Hammer	F	63	50587	8347ryfgvgshf	Y

Example using policy:

```
create or replace masking policy BAR as (val
string) returns string ->
case
  when is_granted_to_invoker_role('SEETOKENS')
  then val
  when current_role('GETREAL')
  then detok_ext_func(val, current_user(),...)
  else '** masked **'
end;
```

Example using SQL outside policy:

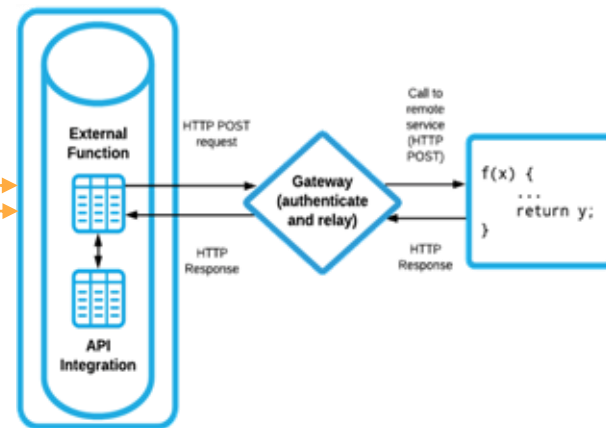
```
SELECT detok_ext_func(T1.phone) AS REAL_PHONE
, T1.GENDER
, T2.ZIP
FROM T1
JOIN T2
ON T2.PHONE = T1.PHONE
;
```

Ingest protected (PII/PHI) data as Externally Tokenized

- Using tokenization provider functionality upstream from Snowflake

De-tokenize for authorized users at query time

- The tokenization provider is called using a Snowflake External Function to de-tokenize data
- For unauthorized users, third-party service is not called
- Can be used in policy or outside



Row Access Policy

RBAC & DAC

Views & UDFs

Column-Level Security

Row Access Policy

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y

Query results

zip_code	name
31675	** masked **



Alex
(Only 31675)

Query results

zip_code	name
28877	** masked **



Morgan
(Only 28877)

Filter rows at query time based on user role and lookup table

- Policy contains condition(s) to allow or filter out rows
- Policy is applied to one or more table, view, or external table in an account
- Dynamically generated predicate filters out rows the user is not authorized to see at query time
- Can be combined with other controls

```
create or replace row access policy F00
as (this_zip varchar) returns boolean ->
  'all_seeing_role' = current_role()
or
  exists (
    select 1 from zip_mapping_table
    where info_reader = current_role()
    and zip_code = this_zip);
```

Data Policies Performance Boost (Memoizable Functions)

Boost performance of complex policies with many mapping table lookups

```
create or replace row function
  allowed_sales_regions() memoizable
returns array as $$ ...
```



Conditional Policy

RBAC & DAC

Views & UDFs

Column-Level Security

Row Access Policy

Conditional Policy

FAMILYNAME	GENDER	AGE	ZIPCODE	PHONE	OPTIN
Romero	F	56	31675	4282478462	Y
Polk	N	52	16971	7565697078	U
Gaines	F	61	28877	4210779144	Y
Gordon	M	36	85115	6474995638	N
Hammer	F	63	50587	5947720813	Y

Query results



All Users

phone	name
MASKED	Polk
4210779144	Gaines
MASKED	Gordon

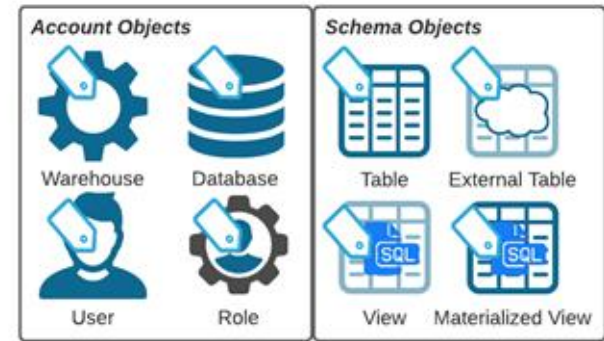
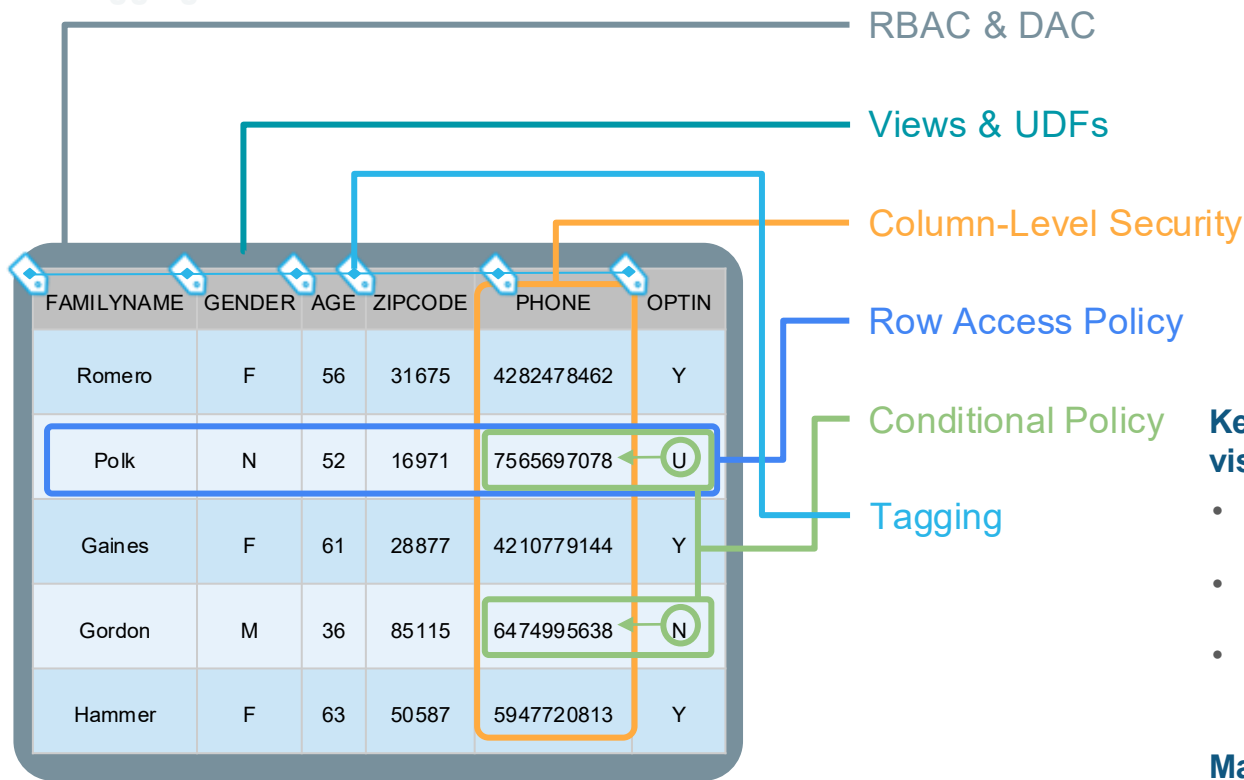
Conditional policy is a type of Column-level Security policy to dynamically mask data at query time in *one column* based on *the value of another*

- Row-level policy for column access
- No change to the stored data
- Same properties as other column-level policies

```
create or replace masking policy BAZ
as (val string, optin string) returns string ->
case
  when optin = 'Y' then val
  else '***MASKED***'
end;
```

```
alter table start_with_a_table modify column PHONE set
masking policy BAZ using (PHONE, OPTIN);
```





Keep track of sensitive data for visibility and compliance

- Assign tags to sensitive columns, tables, external tables, or views
- Easily audit sensitive objects without appropriate security policies
- Assign tags to virtual warehouses, Snowpipe, materialized views, and clustered tables to keep track of resource usage for cost visibility and attribution

Manage tags with flexible administration models

- Centralized tag creation and assignment for centrally managed governance
- Decentralized tag assignment controlled by privileges for object owner-supplied tag value



Tag Based Policy

RBAC & DAC

Views & UDFs

Column-Level Security

Row Access Policy

Conditional Policy

Tagging

Query results



Note that the directly applied Conditional policy overrides the Tag-based policy

phone	age	optin	name
MASKED	88,888,888	U	Polk
4210779144	88,888,888	Y	Gaines
MASKED	88,888,888	N	Gordon

Policy1

Policy2



Column1

Column2

Tag-based policy allows you to assign a policy to a tag instead of a specific object.

- Assign policy based on the metadata tracked in tags
- Assign policy to untagged columns to mask until scanned and curated
- Most specifically-assigned policy for any column wins (column policy overrides tag policy)

```
create tag MASKME;
```

```
create masking policy string1 as (val string) returns string ->
case when current_role() in ('YUP') then val
else '** masked **' end;
```

```
create masking policy numb1 as (val int) returns int ->
case when current_role() in ('YUP') then val
else 88888888 end;
```

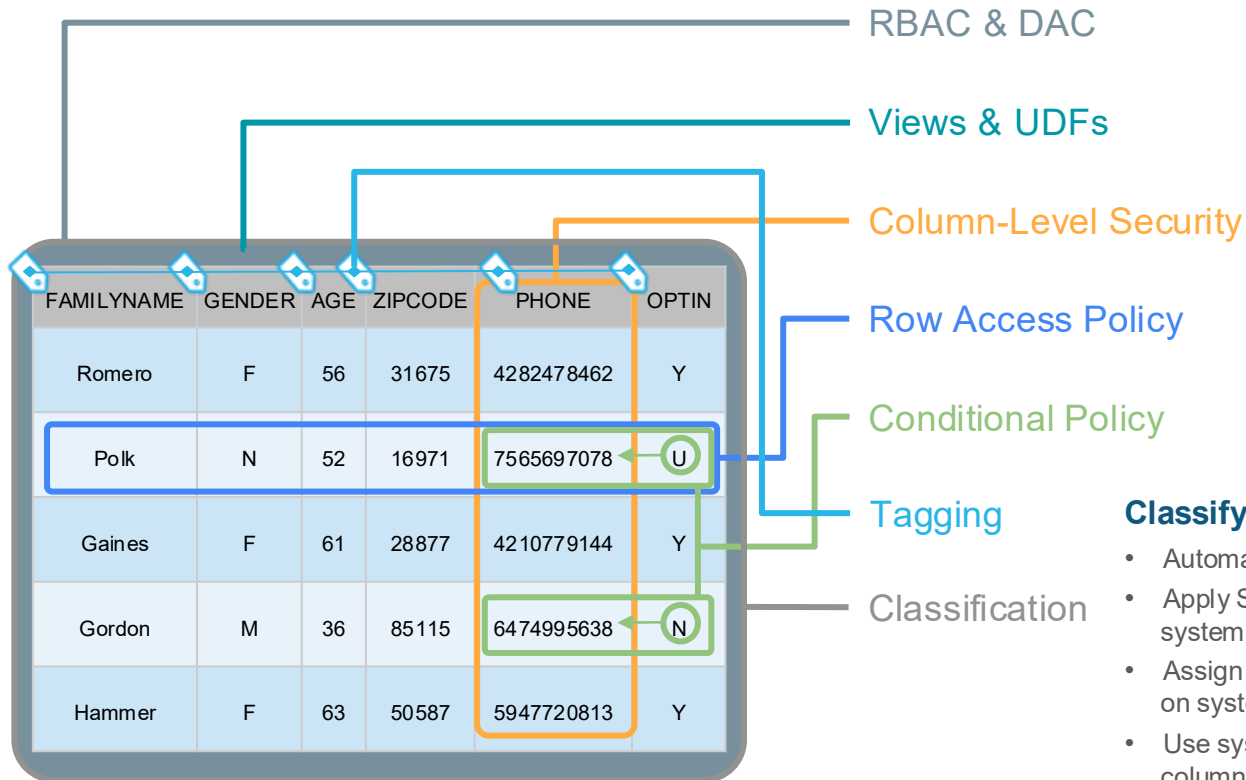
```
alter tag MASKME set masking policy string1,
masking policy numb1;
```

```
alter table start_with_a_table modify column PHONE set tag
MASKME = 'YUP';
```

```
alter table start_with_a_table modify column AGE set tag
MASKME = 'YUP';
```



Classification

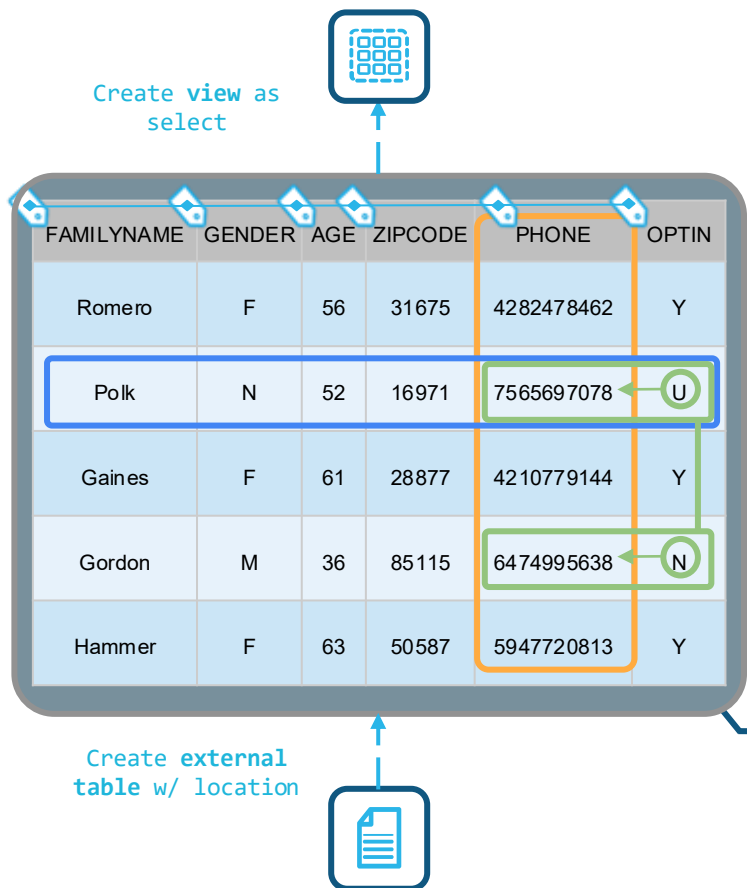


Classify columns containing personal data

- Automatically detect columns with personal data
- Apply Snowflake-defined semantic and privacy category system tags
- Assign data access policies to columns based on system tags
- Use system tags to audit personal data among millions of columns
- Seamlessly integrates across Snowflake's governance capabilities and Data Sharing
- Populate and manipulate tags using third-party GRC, MDM, and other classification solutions



Object Dependencies



Impact analysis as objects are built. Determine data provenance.

- Identify all upstream source tables of a desired view to satisfy regulatory compliance, such as GDPR, CCPA, HIPAA, PCI.
- Proactively identify impacted downstream views before making metadata changes referenced in views and tables.
- Identify source tables and views of desired view to verify trustworthiness and popularity of the source.

Object Dependencies answers difficult questions:

- Where did the data come from?
- What data is impacted by other objects?
- Where is the data located?
- What does the data produce?
- What does the data consume?
- What associations are there between data elements?
- What data sources should we use to develop new customer experience initiatives?
- What data in my enterprise needs to be brought into compliance with industry regulations?
- Where do we have risk exposure that needs to be mitigated?
- Who or what business process changed the data?
- How can data scientists improve confidence in the data they need for advanced analytics?
- What tool or technology made the change?





Snowflake中国微信公众号

Q&A



Snowflake中国微信视频号

